

Threads

2025 Semester 1 SOFTENG 370: Operating Systems
Talia Xu

Lecture 19
2.0.0

Mid-Semester Re-cap

Assessment	Weight	Status
Lab 1	7.5%	Completed
Midterm	20%	Completed
Lab 2	7.5%	Completed
Lab 3	7.5%	
Lab 4	7.5%	
Final	50%	

Course Topics

OS Fundamentals

Process

Scheduling

Memory

Concurrency

File Systems

Security, Device Drivers

Information on Final Exam

June 23rd, 2 pm, 2 hrs 15 mins

Openbook, No calculators permitted

Format: 50 Multiple-choice (tentative)

About 15 questions on the 1st half of the course, 35 questions on the 2nd half

Plans This Week

Collect your teleform sheets.

Tutorial: Midterm Questions

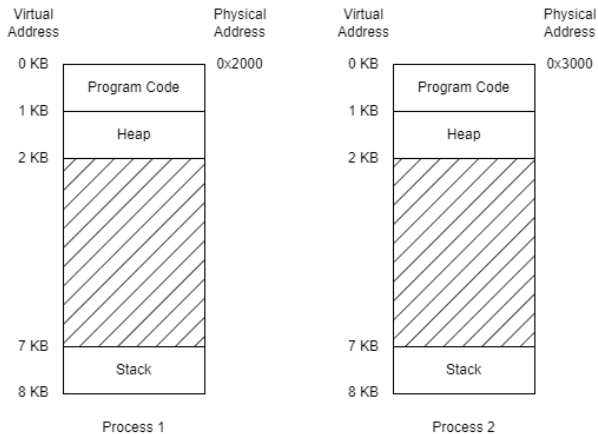
Concurrency: Multi-thread Programming

Game programming: The main thread renders each frame, while a pool of worker threads processes gameplay tasks (AI, physics, audio) in parallel.

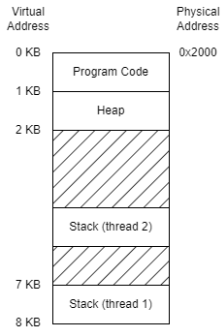
Machine learning: Training launches thousands of lightweight GPU threads per kernel.

Web programming: Server thread pools handle large numbers of concurrent HTTP requests, overlapping I/O and computation to maximize throughput.

Throwback: Process and fork()



Multi-Thread Memory



What are the trade-offs?

Threads are Like Processes with Shared Memory

The same principle as a process, except by default they share memory

They have their own registers, program counter, and stack

They have the same address space, so changes appear in each thread

You need to explicitly state if any memory is specific to a thread (TLS)

One Process Can have Multiple Threads

By default, a process just executes code in its own address space

Threads allow multiple executions in the same address space

They're lighter weight and less expensive to create than processes

They share code, data, file descriptors, etc.

Aside: Concurrency and Parallelism Aren't the Same

Concurrency

Switching between two or more things (can you get interrupted)

Goal: make progress on multiple things

Parallelism

Running two or more things at the same time (are they independent)

Goal: run as fast as possible

Multi-thread programming can achieve both.

We'll be Using POSIX Threads

For Windows, there's a Win32 thread, but we're going to use *UNIX threads

`#include <pthread.h>` — in your source file

`-pthread` — compile and link the pthread library

All the pthread functions have documentation in the `man` pages

You Create Threads with `pthread_create`

```
int pthread_create(pthread_t* thread,  
                  const pthread_attr_t* attr,  
                  void* (*start_routine)(void*),  
                  void* arg);
```

thread creates a handle to a thread at pointer location

attr thread attributes (NULL for defaults, more details later)

start_routine function to start execution

arg value to pass to start_routine

returns 0 on success, error number otherwise

(contents of *thread are undefined)

Creating a thread

```
#include <pthread.h>
#include <stdio.h>

void* run(void *arg) {
    printf("%s\n", (char *) arg);
    return NULL;
}

int main() {
    pthread_t p1, p2;
    pthread_create(&p1, NULL, &run, "A");
    pthread_create(&p2, NULL, &run, "B");
    printf("In main\n");
}
```

The wait Equivalent for Threads — Join

```
int pthread_join(pthread_t thread,  
                 void** retval)
```

thread wait for this thread to terminate (thread must be joinable)

retval stores exit status of thread (set by pthread_exit) to the location pointed by *retval. If cancelled returns PTHREAD_CANCELED. NULL is ignored.

returns 0 on success, error number otherwise

Only call this one time per thread!

Multiple calls on the same thread leads to undefined behavior

Previous Example that Waits Properly

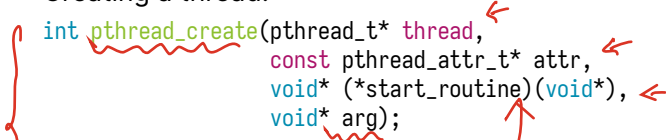
```
void* run(void *arg) {  
    printf("%s\n", (char *) arg);  
    return NULL;  
}  
  
int main() {  
    pthread_t p1, p2;  
    pthread_create(&p1, NULL, &run, "A");  
    pthread_create(&p2, NULL, &run, "B");  
    pthread_join(p1, NULL);  
    pthread_join(p2, NULL);  
    printf("In main\n");  
}
```

Recap for Yesterday

Threads are like processes with shared memory.

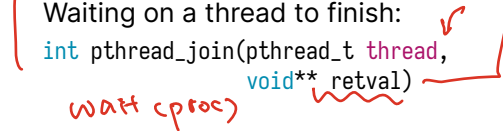
Creating a thread:

```
int pthread_create(pthread_t* thread,
                  const pthread_attr_t* attr,
                  void* (*start_routine)(void*),
                  void* arg);
```



Waiting on a thread to finish:

```
int pthread_join(pthread_t thread,
                 void** retval)
```



wait (proc)

An Array Example

I want to create a 1×5 int array and fill it with the elements {10, 20, 30, 40, 50}.

I want to create a thread to handle this task.

Where do I allocate this memory? - heap

Which thread allocates the memory?

↳ 2 threads

main — workers

An Array Example - Main Thread Allocates

```
#define N 5
void *array_ex(void *arg)
{
    int *a = arg;
    for (int i = 0; i < N; ++i)
        a[i] = (i + 1) * 10;
    return NULL;
}

int main(void)
{
    pthread_t p1;
    int *array = malloc(N * sizeof(int)); (heap)
    thread_create(&p1, NULL, array_ex, array);
    pthread_join(p1, NULL);
    free(array);
    return 0;
}
```

Handwritten annotations in red:

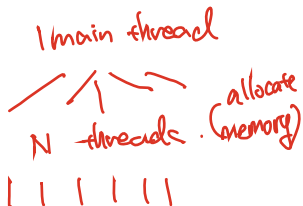
- A checkmark above `#define N 5`.
- A circle around `(void *arg)` in the function signature of `array_ex`.
- A box around `int` in `int *a = arg;`.
- A circle around `return NULL;` in the `array_ex` function.
- An arrow pointing from the `array_ex` function to the `array_ex` argument in `thread_create`.
- An arrow pointing from the `(heap)` note to the `malloc` function.
- An arrow pointing from the `array` variable in `thread_create` to the `array` argument in `thread_create`.
- An arrow pointing from the `array` argument in `thread_create` to the `array` argument in `pthread_join`.
- An arrow pointing from the `array` argument in `pthread_join` to the `array` argument in `free`.
- A red number `5` above the `N` in `malloc(N * sizeof(int))`.
- An arrow pointing from the `5` to the `N` in `malloc`.

An Array Example - Worker Thread Allocates

```
#define N 5
void *make_array(void *arg)
{
    int *a = malloc(N * sizeof(int)); ←
    for (int i = 0; i < N; ++i)
        a[i] = (i + 1) * 10; ←
    return a;
}
int main(void)
{
    pthread_t tid;
    int *array = NULL; ←
    pthread_create(&tid, NULL, make_array, [NULL] != 0;
    pthread_join(tid, (void **)&array);
    free(array);
    return 0;
}
```

allocate in
main thread.

free in
main thread



(pointer) → value.
[address - pointer]

Previous Example that Waits Properly

```
void* run(void *arg) {  
    int* retval = malloc(sizeof(int));  
    *retval = 42;  
    return retval;  
}  
int main() {  
    pthread_t p1;  
    void* ret_val;  
  
    pthread_create(&p1, NULL, &run, "A");  
    pthread_join(p1, &ret_val);  
  
    int result = *(int*)ret_val;  
    free(ret_val);  
}
```

Ending a Thread Early (Think of exit)

```
void pthread_exit(void *retval);
```

retval return value passed to function that calls pthread_join

Note: start_routine returning is equivalent of calling pthread_exit
Think of the difference between returning from main and exit

pthread_exit is called implicitly when the start_routine of a thread returns

Previous Example that Waits Properly

```
void* run(void *arg) {  
    int* retval = malloc(sizeof(int));  
    *retval = 42;  
    return retval; // pthread_exit(retval) is called;  
}  
int main() {  
    pthread_t p1;  
    void* ret_val;  
  
    pthread_create(&p1, NULL, &run, "A");  
    pthread_join(p1, &ret_val);  
  
    int result = *(int*)ret_val;  
    free(ret_val);  
}
```

Detached Threads

- *Joinable* threads (the default) wait for someone to call `pthread_join` then they release their resources
- *Detached* threads release their resources when they terminate

```
int pthread_detach(pthread_t thread);
```

`thread` marks the thread as detached
returns 0 on success, error number otherwise

Calling `pthread_detach` on an already detached is undefined behavior

Detached Threads Aren't Joined

```
void* run(void*) {  
    printf("In run\n");  
    return NULL;  
}
```

```
int main() {  
    pthread_t thread;  
    pthread_create(&thread, NULL, &run, NULL);  
    pthread_detach(thread);  
    printf("In main\n");  
    return 0;  
}
```

main thread does not
wait for worker thread
to complete.

This code just prints "In main", why?

pthread_exit in main Waits for All Detached Threads to Finish

```
void* run(void*) {  
    printf("In run\n");  
    return NULL;  
}
```

```
int main() {  
    pthread_t thread;  
    pthread_create(&thread, NULL, &run, NULL);  
    pthread_detach(thread);  
    printf("In main\n");  
    pthread_exit(NULL);  
}
```

(return 0 ; // exit code),



You Can Use Attributes To Get/Set Thread Variables

```
size_t stacksize;  
pthread_attr_t attributes;  
pthread_attr_init(&attributes);  
pthread_attr_getstacksize(&attributes, &stacksize);  
printf("Stack size = %i\n", stacksize);  
pthread_attr_destroy(&attributes);
```

Running this should show a stack size of 8 MiB (on most Linux systems)

priority .
scheduling .

Multithreading Models

Where do we implement threads?

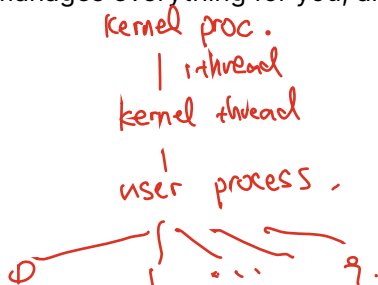
We can either do user or kernel threads

User threads are completely in user-space

Kernel doesn't treat your threaded process any differently

Kernel threads are implemented in kernel-space

Kernel manages everything for you, and can treat threads specially



Thread Support Requires a Thread Table

Similar to the process table we saw previously

It could be in user-space or kernel-space (depending)

For user threads, there also needs to be a run-time system to determine scheduling

In both models each process can contain multiple threads

We Could Avoid System Calls, or Let a Thread Block Everything

For pure user-level threads (again, no kernel support):

- Very fast to create and destroy, no system call, no context switches
- One thread blocks, it blocks the entire process (kernel can't distinguish)

For kernel-level threads: ←

- Slower, creation involves system calls
- If one thread blocks, the kernel can schedule another one

All Threading Libraries You Use Run in User-mode

The thread library maps user threads to kernel threads

Many-to-one: threads completely implemented in user-space
the kernel only sees one process

One-to-one: one user thread maps directly to one kernel thread
the kernel handles everything

Many-to-many: many user-level threads map to many kernel level threads

Many-to-one is Pure User-space Implementation

It's fast (as outlined before) and portable

It doesn't depend on the system, it's just a library

Drawbacks are that one thread blocking causes all threads to block

Also, we cannot execute threads in parallel

The kernel will only schedule a process to run

One-to-one Just Uses the Kernel Thread Implementation

There's just a thin wrapper around the system calls to make it easier to use

Exploits the full parallelism of your machine

The kernel can schedule multiple threads simultaneously

We do however need to use a slower system call interface,
and we lose some control

Typically, this is the actual implementation used, we'll assume this for Linux

Many-to-many is a Hybrid Approach

The idea is that there are more user-level threads than kernel-level threads
Cap the number of kernel-level threads to the number we could run in parallel

We can get the most out of multiple CPUs and reduce the number of system calls

However, this leads to a complicated thread library (Java Virtual Threads)
Depending on your mapping luck, you may block other threads



Threads Complicate the Kernel



How should fork work with a process with multiple threads?

Copy all threads to the new process, in whatever state they're in?

How would this get out of hand?

Linux only copies the thread that called fork into a new process

If it hits `pthread_exit` it'll always exit with status 0

(at least as far as I can tell)

There's `pthread_atfork` (not covered in this course) to control what happens

Signals are Sent to a Process

Which thread should receive a signal? all of them?

Linux will just pick one random thread to handle the signal

Makes concurrency hard, any thread could be interrupted

Instead of Many-to-many, You Can Use a Thread Pool

The goal of many-to-many thread mapping is to avoid creation costs

A thread pool creates a certain number of threads and a queue of tasks
(maybe as many threads as CPUs in the system)

As requests come in, wake them up and give them work to do

Reuse them, when there's no work, put them to sleep